VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018, Karnataka



NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System

A Report submitted in partial fulfillment of the requirements for the Course

PROJECT WORK – 2

(Course Code: 24AM7PWPW2)

In the Department of

MACHINE LEARNING

(UG Program: B.E. in Artificial Intelligence and Machine Learning)

By

Aryan Kumar	1BM23AI037
Roshanth V	1BM23AI155
S S Gokula Swamy	1BM23AI158
Somanath S D	1BM23AI187

Semester & Section: VII – 'A'&'C'

Under the Guidance of

Prof. Varsha R

Department of Machine Learning, BMSCE, Bengaluru – 19



DEPARTMENT OF MACHINE LEARNING

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)
P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

November – 2026

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)
P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF MACHINE LEARNING



CERTIFICATE

This is to certify that Aryan Kumar bearing USN: 1BM23AI037, Roshanth V bearing USN: 1BM23AI155, S S Gokula Swamy bearing USN: 1BM23AI158, and Somanath S D bearing USN: 1BM23AI187 have satisfactorily completed the Course – Project Work – 2 (Course Code: 24AM7PWPW2) with the title "NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System" in partial fulfillment of the academic curriculum requirements of the VII Semester UG Program – B.E. in Artificial Intelligence and Machine Learning in the Department of Machine Learning, BMSCE, an Autonomous Institute, affiliated to Visvesvaraya Technological University, Belagavi during November 2026. The base work considered for the presentation is used for academic purposes and has not been used in its original form elsewhere for the award of any degree.

Student Name (USN)	**Student Signature**
Aryan Kumar (1BM23AI037) Roshanth V (1BM23AI155) S S Gokula Swamy (1BM23AI158) Somanath S D (1BM23AI187)	

Signature of the Supervisor	**Signature of the Head**
Prof. Varsha R Assistant Professor, Dept. of MEL, BMSCE	**Dr. Dakshayini Bhat** Professor & Head, Dept. of MEL, BMSCE

External Examination

Examiner Name and Signature

1.

2.

DECLARATION

We hereby declare that the project entitled “NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System” is a project work carried out by us as part of Project Work - 2 (24AM7PWPW2) in the Department of Machine Learning, B.M.S. College of Engineering, Bengaluru, during the academic year 2026-2027, under the guidance of Prof. Varsha R.

We further declare that the technical descriptions and results presented in this report are based on the project repository and the development/evaluation records available at the time of report preparation. All external models, datasets, libraries, standards and publications used have been acknowledged in the references. Results are reported with their documented limitations; no unverified outcome is represented as a validated system capability.

Place: Bengaluru

Date: 31-10-2026

Signatures

ACKNOWLEDGEMENT

We express our sincere gratitude to B.M.S. College of Engineering and the Department of Machine Learning for providing the academic environment and facilities required to undertake this project. We are grateful to our project guide, Prof. Varsha R, for her guidance, feedback and encouragement throughout the development of NutriSense.

We acknowledge the contributions of the open-source research and engineering communities whose work forms the foundation of the computer-vision and machine-learning components used in this project. In particular, the project builds upon research and software associated with Segment Anything Model 2.1, SigLIP 2, MoGe-2, DINOv2, PyTorch, OpenCV, Transformers, FastAPI and React. We also acknowledge the public nutrition and programme documents and food-composition references cited in the project data and documentation.

Finally, we thank all individuals who provided technical discussions, project feedback and assistance during research, implementation, data collection planning, testing and documentation.

ABSTRACT

NutriSense is an AI-based meal-analysis prototype that estimates the edible portion mass of food from an image and assesses selected energy and protein requirements for configured school-meal categories. It addresses a monitoring gap: a record that a meal was served does not establish the nutritional value of the portion on an individual plate. The system therefore explores a vision-based workflow that produces an auditable estimate while exposing image-quality and evidence limitations.

The implementation combines a Python FastAPI backend with a React and TypeScript frontend. Its analysis pipeline validates the image and context, detects a known-size ArUco marker where available, segments food using SAM 2.1, matches dish crops against a SigLIP 2 gallery, estimates depth with MoGe-2, and derives portion mass through geometric or eligible learned estimation paths. Monte Carlo sampling propagates portion and composition uncertainty into energy and protein estimates. A structured response records verdicts, reasons, failures, assumptions, coverage and model/policy provenance.

On the documented WED evaluation setup, the deployable mass-estimation head reports 15.034 g development MAE and 14.667 g locked-evaluation MAE. These are portion-mass results, not measures of calorie or protein accuracy. The recorded nominal 90% prediction intervals under-cover, and the project readiness notes unresolved multi-item, nutrition-transfer, calibration and production-validation work. NutriSense is therefore a research prototype and decision-support workflow, not a certified weighing device or autonomous compliance authority.

Keywords: Artificial Intelligence, Machine Learning, Computer Vision, Food Image Analysis, SAM 2.1, SigLIP 2, Monocular Depth Estimation, Portion Mass Estimation, Monte Carlo Uncertainty, PM POSHAN.

TABLE OF CONTENTS

Declaration
Acknowledgement
Abstract
Table of Contents
List of Figures
List of Tables
CHAPTER 1 INTRODUCTION
1.1 Overview
1.2 Scope
1.3 Existing System
1.4 Proposed System
CHAPTER 2 PROBLEM STATEMENT
2.1 Problem Statement
2.2 Motivation
2.3 Objectives
CHAPTER 3 LITERATURE SURVEY
3.1 Analysis of Gaps, Project Progress and Completed Activities
3.1.1 Image Segmentation
3.1.2 Dish Recognition and Vision-Language Embeddings
3.1.3 Monocular Depth and Scale Recovery
3.1.4 Learned Portion-Mass Estimation
3.1.5 Probabilistic Nutrient Estimation and Decision Rules
3.1.6 Gaps Identified and Current Project Progress
3.1.7 Completed Activities Recorded in the Repository
3.2 Justification of the Selected Problem
3.3 Project Contribution to Society and Sustainable Development Goals
CHAPTER 4 SYSTEM REQUIREMENT SPECIFICATION
4.1 Functional Requirements
4.2 Non-functional Requirements
4.3 Hardware Requirements
4.4 Software Requirements
CHAPTER 5 SYSTEM DESIGN
5.1 System Architecture
5.2 Methodology
5.2.1 Image Capture and Input Validation
5.2.2 Metric Scale Anchor Using ArUco
5.2.3 Food Segmentation
5.2.4 Dish Classification
5.2.5 Depth Estimation and Portion Mass
5.2.6 Nutrient Composition and Monte Carlo Estimation
5.2.7 Nutritional Adequacy Assessment
5.3 Detailed Design

5.3.1 Class Diagram
5.3.2 Activity Diagram
5.3.3 Use Case Diagram
CHAPTER 6 IMPLEMENTATION
6.1 Overview of Tools and Technologies Used
6.2 Details of the Modules
6.2.1 API and Request Handling Module
6.2.2 Configuration and Data Management Module
6.2.3 Input Gates and Capture Lint Module
6.2.4 Scale Anchor and Camera Geometry Module
6.2.5 Segmentation and Classification Modules
6.2.6 Depth and Portion Estimation Modules
6.2.7 Nutrition Monte Carlo and Compliance Modules
6.2.8 Analysis Contract and Frontend Module
6.3 Challenges Encountered and Strategies Used to Address Them
CHAPTER 7 TESTING AND RESULTS
7.1 TESTING
7.1.1 Unit Testing
7.1.2 Integration Testing
7.1.3 System Testing
7.2 RESULTS
7.2.1 Portion-Mass Estimation Results on WED
7.2.2 Uncertainty Calibration
7.2.3 Nutrition Estimation and Validation
7.2.4 Real-capture Pilot and Multi-item Findings
7.2.5 Overall Readiness Assessment
CHAPTER 8 PROJECT MANAGEMENT
8.1 Gantt Chart
8.2 Team Roles and Responsibilities in Project Execution
8.3 Challenges Encountered and Mitigation Strategies
CONCLUSION & FUTURE WORK
Future Work
LEARNING OUTCOMES
REFERENCES
APPENDIX A: SNAPSHOTS

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	NutriSense problem context and output workflow	
Figure 1.2	Existing and proposed system comparison	
Figure 5.1	High-level system architecture	
Figure 5.2	End-to-end analysis pipeline	
Figure 5.3(a)	Conceptual class and module diagram	
Figure 5.3(b)	Activity diagram for meal analysis	
Figure 5.3(c)	Use case diagram	
Figure 6.1	Backend and frontend implementation layout	
Figure 6.2	Nutrition assessment and uncertainty reporting	

LIST OF TABLES

Table No.	Title	Page No.
Table 1.2	Project scope and boundaries	
Table 1.1	Comparison of conventional and proposed approaches	
Table 3.1	Technical gaps and NutriSense response	
Table 3.2	Project contribution to the Sustainable Development Goals	
Table 4.1	Functional requirements	
Table 4.2	Non-functional requirements	
Table 4.3	Hardware requirements and evidence	
Table 5.1	Main analysis pipeline stages and responsibilities	
Table 5.2	Nutritional compliance thresholds encoded in NutriSense	
Table 5.3	Analysis response contract (selected fields)	
Table 6.1	Software and technology stack	
Table 6.2	Implementation challenges and mitigation strategies	
Table 7.1	Project evaluation metrics and documented evidence	
Table 8.1	Indicative project work plan / Gantt chart	
Table 8.2	Team roles and responsibilities	

CHAPTER 1

INTRODUCTION

1.1 Overview

Food and nutrition programmes are intended not only to make meals available but also to provide meals that meet defined nutritional requirements. In school settings, meal quality is relevant to children's health, well-being and ability to participate in education. Operational monitoring commonly involves menu records, ingredient allocations, preparation records and physical inspection. These sources are useful, but they do not always quantify the actual cooked portion served on an individual plate.

NutriSense investigates whether computer vision and machine learning can assist this monitoring process. A user captures an image of a served meal, supplies basic context such as the menu day and student class band, and submits the image for analysis. The system attempts to identify the supported dish, isolate the food region, infer its scale and portion mass, estimate energy and protein from component-level composition data, and compare probability distributions of the estimated nutrients with configured minimum requirements.

The project is a decision-support prototype. Image-based estimation is inherently uncertain because a two-dimensional image does not directly reveal food depth, density, hidden ingredients, serving history or recipe composition. NutriSense therefore includes an evidence-quality and coverage mechanism. It is designed to communicate when scale is measured, when a value is estimated from a prior, when inputs are out of scope, and when the evidence is not adequate for a confident result.

At the software level, NutriSense is implemented as a Python FastAPI backend and a React 18/TypeScript frontend. The backend coordinates model inference, policy checks, data access and a typed analysis response. The model components include SAM 2.1 for segmentation, SigLIP 2 for gallery-based dish recognition, MoGe-2 for monocular depth, and a DINOv2-based learned weight-estimation path trained on WED data. Nutrient calculations and compliance rules are implemented as explicit computational stages rather than being delegated to a single opaque classifier.

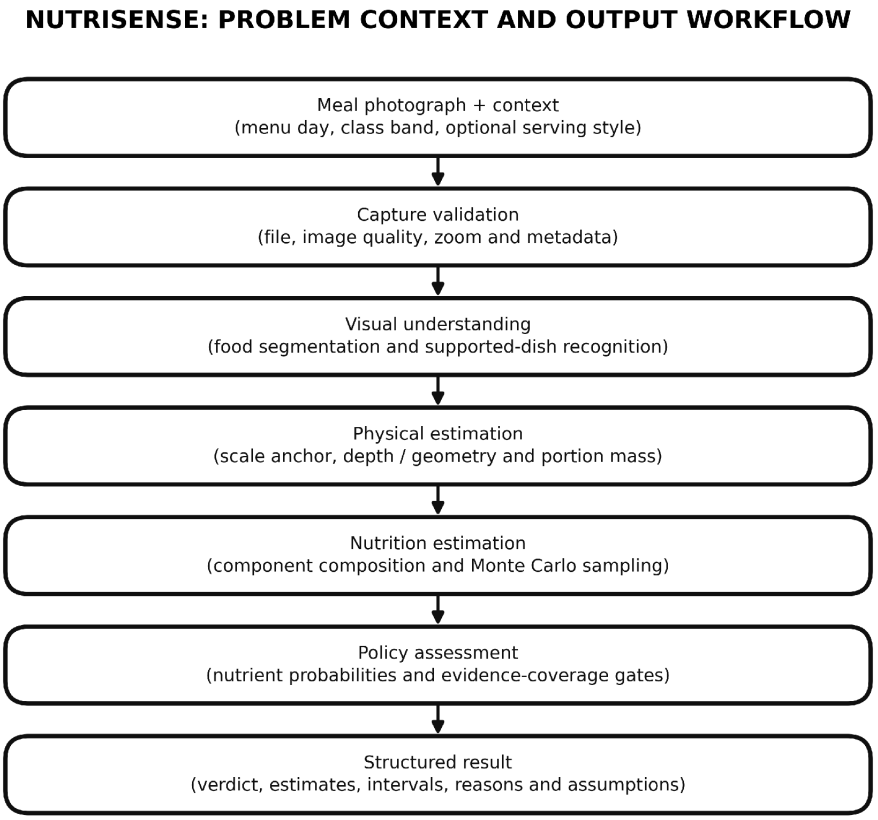


Figure 1.1: NutriSense problem context and output workflow

1.2 Scope

The scope of NutriSense is limited to image-based estimation and nutritional adequacy assessment for supported meal categories and configured class bands. The codebase contains a menu configuration and policy standards for selected school meal cases, along with broader experimental/off-menu model mappings. The API requires a day and class band and optionally accepts a serving style. The system uses the menu configuration as the source of truth for supported dish names and day availability.

Table 1.2: Project scope and boundaries

Included within scope	Outside the validated scope / constraints
Image upload through a browser interface and FastAPI endpoint.	Clinical diagnosis, individual medical advice or disease prediction.
Food-region segmentation, dish classification and geometric/learned portion-mass estimation.	Certified weighing-device functionality or guaranteed exact mass from one image.
Energy and protein estimates from configured composition rows.	Validated full macronutrient panel; repository states fat/carbohydrate traces are not supported by the current local composition table.
PM POSHAN-style energy/protein threshold assessment for configured bands.	Automatic government audit, beneficiary identity verification or final administrative compliance decision.
Explicit uncertainty, evidence coverage, assumptions and abstention paths.	Guarantee of calibrated 90% intervals; existing calibration does not meet the documented coverage/width gate.
Research evaluation and local prototype deployment.	Production-scale multi-user deployment; operational budgets and sustained concurrency are not established.

The project is also bounded by its capture protocol. The measured-scale route is designed around a printed 60.0 mm ArUco marker placed beside the vessel on the same plane. Digital zoom must be disabled, the original image metadata should be preserved, the marker must be visible at adequate pixel size, and the tray must be fully inside the frame. These controls improve repeatability but require cooperation from the person collecting the image.

1.3 Existing System

Conventional meal monitoring can use paper-based menu registers, ingredient allocation schedules, kitchen-level production records, manual weighing, visual inspection and periodic nutrition surveys. Each method captures a different aspect of programme operation. Ingredient allocation can express intended quantities, whereas weighing can provide direct mass measurements. Manual calculations can estimate nutrient values when the recipe and composition information are known.

However, these methods have practical limitations. Manual weighing each served portion can be time-consuming. Visual inspection alone cannot reliably infer mass or nutrient content. A menu entry does not establish the composition of a particular cooked plate. Recipe variation, water absorption, cooking yield, oil quantity and serving density can affect the relationship between raw ingredient allocations and cooked plate mass. A monitoring system should also communicate uncertainty rather than translating every incomplete observation into a binary decision.

Table 1.1: Comparison of conventional and proposed approaches

Aspect	Conventional/manual approach	NutriSense approach
Observation	Menu, record, manual inspection or direct weighing.	Photograph-based capture with contextual inputs.
Portion mass	Scale measurement or visual judgement.	Geometric estimate with reference marker, or learned WED mass head for eligible no-card cases.
Food identity	Known from menu/observer.	Gallery-based recognition using image embeddings; unsupported cases can abstain.
Nutrient calculation	Manual lookup and arithmetic.	Component mass multiplied by configured per-100 g nutrient composition, with uncertainty sampling.
Decision	Human interpretation of records and measurements.	Probability and coverage thresholds generate PASS, FAIL or BORDERLINE, subject to gates.
Traceability	Depends on record-keeping practice.	Structured reasons, assumptions, model versions, policy digest and provenance fields.

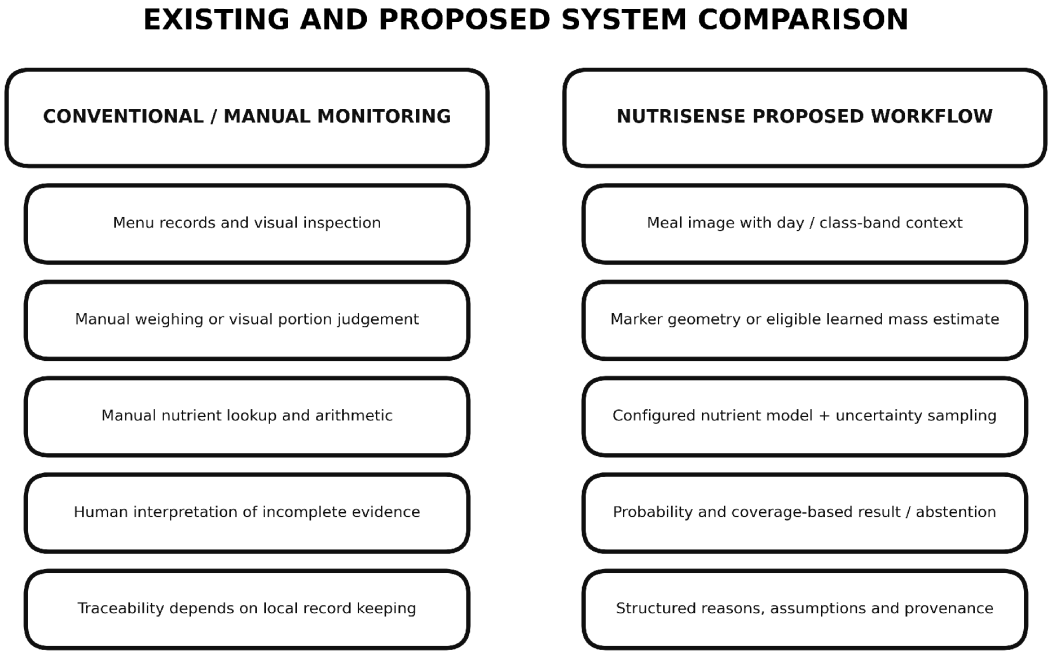


Figure 1.2: Existing and proposed system comparison

1.4 Proposed System

NutriSense proposes an integrated image-analysis workflow that combines multiple specialised models and deterministic policy logic. The system begins with an image and context fields, performs capture linting, and then follows a sequence of model and computational stages. A physical reference card supplies a known dimension for image scale. Food segmentation isolates a candidate food region. Dish classification maps the appearance to a supported dish label. Monocular depth and camera geometry are used to estimate food

geometry, while dish configuration supplies component identities, nominal shares and density values. Where the reference card is absent, the implementation may use a trained weight head for a mapped dish when a compatible checkpoint is available; geometry priors are not silently substituted for that learned path when the head is required but unavailable.

After estimating portion mass, NutriSense propagates uncertainty through Monte Carlo sampling. The nutrient engine samples uncertainties in mass and composition, computes energy and protein samples, and summarises their distributions. The compliance module

computes the probability that each mandatory nutrient reaches its configured minimum. A result is then determined using both nutrient probability and evidence coverage. A structured reason is attached when evidence is estimated, degraded or insufficient.

The proposed system therefore differs from a basic image classifier. It aims to make each stage inspectable and to prevent a visually plausible result from being presented without its evidence quality. Its current limitations, including incomplete validation of nutrition transfer and multi-item analysis, are documented in Chapter 7.

CHAPTER 2

PROBLEM STATEMENT

2.1 Problem Statement

There is a need for a repeatable, low-friction method to estimate the portion mass and selected nutritional properties of prepared meals using accessible image capture, while making the reliability of the estimate explicit. In a school-meal monitoring context, the key challenge is not simply recognising the dish. A useful assessment requires the system to estimate how much food is present, relate the mass to an appropriate composition model, account for uncertainty and compare the resulting nutrient distribution with a policy threshold.

The technical problem can be expressed as follows: given an RGB image I, menu-day context d, student class band b and optional serving-style metadata s, estimate a food/dish representation, portion mass M and nutrient vector N = (E, P), where E denotes energy in kilocalories and P denotes protein in grams. Then estimate the probabilities P(E >= E_min(b)) and P(P >= P_min(b)) and produce a policy result only when the capture and evidence coverage conditions are met. The system must also identify inputs for which this calculation is not defensible and abstain or return cannot_verify/out_of_scope rather than inventing a value.

The problem is challenging because image pixels are not direct measurements of grams. Perspective, camera calibration, occlusion, bowl geometry, food texture, depth estimation, density assumptions, recipe composition and image quality all affect the result. For this reason, the project treats portion mass, nutrient composition and compliance as separate but linked estimation and decision stages.

2.2 Motivation

The motivation is to support evidence-based observation of meal adequacy. In many food-programme settings, records may establish which menu item was planned or served, but assessing the nutritional content of an individual plate requires additional measurement. A computer-vision workflow may help reduce repetitive manual calculation and enable structured data collection, especially where basic image-capture devices are available.

A second motivation is the need for transparent uncertainty. An automated system should not imply that a single photograph provides a laboratory measurement. NutriSense therefore uses a reference marker, data-quality linting, uncertainty propagation, explicit prior tiers and policy coverage thresholds. It also records assumptions such as estimated recipe shares and nutrient-table rows.

The project is aligned with the practical use of machine learning for a public-interest problem. It combines model development with data provenance, evaluation discipline, interpretable output and ethical image-capture rules. It also creates a foundation for future studies using independently weighed meals and better-calibrated composition data.

2.3 Objectives

- Develop a web-based system for uploading a meal photograph and supplying the required contextual metadata.
- Implement capture-quality checks, including image resolution, digital zoom, marker visibility and valid menu-day/class-band inputs.
- Use computer-vision models to segment food regions and recognise supported dishes.
- Estimate physical scale and food geometry from a known-size ArUco marker, camera information and monocular depth, with clearly identified prior/learned alternatives when measured scale is unavailable.
- Estimate portion mass and, where configured, decompose meals into component-level mass contributions.
- Estimate energy and protein using a traceable food-composition table and propagate input uncertainty through Monte Carlo sampling.
- Assess nutritional adequacy using probability and evidence-coverage thresholds defined in the project policy configuration.
- Return a structured and auditable analysis with reasons, failures, assumptions, model versions, policy information and degradations.
- Evaluate the system using unit, integration, frontend and model evaluation procedures, and document current limitations honestly.

CHAPTER 3

LITERATURE SURVEY

NutriSense integrates ideas from image segmentation, vision-language representation learning, monocular depth estimation, fiducial marker geometry, learned mass regression and probabilistic decision systems. The literature and model references below are selected because they correspond directly to components identified in the repository. The purpose of this chapter is to explain the relevance of these methods to the project architecture, not to claim that any individual paper validates the complete NutriSense application.

3.1 Analysis of Gaps, Project Progress and Completed Activities

Author name	Research paper title	Their work	Research gap
Ravi et al. (2024)	SAM 2: Segment Anything in Images and Videos	Introduces promptable image and video segmentation using visual prompts and memory-based tracking.	General segmentation does not guarantee food-only masks; food, vessel and background separation needs domain-specific filtering and validation.
Tschannen et al. (2025)	SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features	Develops vision-language representations for semantic understanding, localization and dense visual features.	A general encoder is not a validated school-meal classifier; gallery matching depends on representative reference images and dish mappings.
Wang et al. (2025)	MoGe-2: Accurate Monocular Geometry with Metric Scale and Sharp Details	Estimates scene geometry and depth from a single image.	Monocular geometry alone does not establish a reliable physical scale for a food portion. NutriSense adds a known-size ArUco reference and fallback handling.
Oquab et al. (2023)	DINOv2: Learning Robust Visual Features without Supervision	Learns general-purpose visual features through self-supervised training.	Generic features do not directly predict food mass; a task-specific mass head and representative weighed examples are needed.
Garrido-Jurado et al. (2014)	Automatic generation and detection of highly reliable fiducial markers under occlusion	Presents fiducial marker generation and detection methods for robust visual localization.	Marker detection supports geometric calibration but does not itself solve food segmentation, depth ambiguity or nutrient estimation.

3.1.1 Image Segmentation

The Segment Anything Model 2 (SAM 2) family provides promptable segmentation for images and videos. NutriSense uses the SAM 2.1 Hiera-Large model through the Transformers interface. Its implementation queries the image centre using a point prompt and evaluates multiple candidate masks. Candidate selection incorporates solidity, area

constraints and containment relationships rather than selecting solely by the model's raw predicted IoU score. If the centre-point result is absent, too small or too large, the implementation makes a centre-box prompted retry. This is important because a segmentation mask that captures only a rim arc or a small patch of food would distort all downstream geometric estimates. [1].

The current implementation also includes a multi-vessel route. It samples a 3-by-3 grid of prompts inside a coarse box, selects candidate masks, removes near-duplicates using intersection-over-union (IoU), filters by area and limits the number of detected vessels to four. The repository notes that this route remains a research area: on real canteen photographs, the item-mask areas can spill into surrounding context and the same food component may be assigned to multiple vessels in some diagnostic frames.

3.1.2 Dish Recognition and Vision-Language Embeddings

SigLIP 2 is a vision-language encoder designed to produce semantically useful visual representations. NutriSense uses a SigLIP 2 base patch-16 224 checkpoint to embed a food crop and compare it with a gallery of reference dish images. The selected class is therefore gallery-based rather than a conventional closed-set classifier trained only within this repository. This approach allows the system to manage its dish vocabulary through reference images and mappings. However, the performance depends on gallery coverage, visual similarity, image capture conditions and the chosen abstention threshold. Recognition accuracy on a controlled gallery must not automatically be interpreted as robustness to arbitrary canteen photographs. [2].

3.1.3 Monocular Depth and Scale Recovery

Monocular depth estimation attempts to infer scene depth from a single RGB image. NutriSense integrates MoGe-2 for dense depth and geometry cues. A monocular model alone does not guarantee a reliable physical metric scale. The project therefore uses the known physical size of a fiducial marker to establish scale and calibrate the model's depth output. The marker is detected using OpenCV ArUco functions, with pose estimation based on camera intrinsics derived preferentially from EXIF 35 mm-equivalent focal length, then EXIF focal length and an assumed sensor width, or a fallback focal length. [3], [5].

The marker establishes a measured anchor only if the configured marker identity is detected, its size is sufficient, and its quiet-zone check passes. If the marker cannot be used, the anchor

stage falls back to a prior tier. The implementation deliberately does not use plate diameter as a scale fallback. This design makes the source of metric scale explicit and avoids treating an unknown plate size as a physical calibration object.

3.1.4 Learned Portion-Mass Estimation

The repository includes a DINOv2-feature-based learned weight head trained and evaluated using the WED dataset. It predicts mass for mapped food classes and provides an alternative for eligible no-reference-card cases. This route is distinct from geometric estimation: it relies on learned appearance-to-mass relationships rather than a physically measured scale. The backend loads the head lazily when the checkpoint exists and uses it for mapped dishes under the configured conditions. A missing or failed checkpoint is not treated as proof of a valid learned estimate. Where the required head is unavailable, the pipeline produces a structured failure for that path rather than silently switching to a misleading geometry measurement. [4].

3.1.5 Probabilistic Nutrient Estimation and Decision Rules

Food composition tables provide nutrient density values, usually expressed per 100 g edible portion. For an estimated component mass m_i and table density q_i, the point contribution is proportional to m_i q_i / 100. In NutriSense, these inputs are not treated as exact. The Monte Carlo engine samples uncertainty in the portion and nutrient-table values, calculates energy and protein samples and then aggregates samples across detected meal components. The compliance engine evaluates the probability that a mandatory nutrient meets its minimum, with additional coverage requirements to prevent low-quality estimates from receiving definitive labels. [6], [7].

3.1.6 Gaps Identified and Current Project Progress

Table 3.1: Technical gaps and NutriSense response

- A photograph has no inherent physical scale.: Known-size ArUco reference marker and camera pose/depth calibration. — Measured tier implemented; prior tier remains uncertain and requires careful disclaimer.
- Food mask can include plate, rim or table.: SAM 2.1 candidate filtering, containment logic and box-prompt fallback. — Synthetic cases validated; real-photo segmentation robustness still needs broader evidence.

- Dish name alone does not determine portion mass.: Geometry pipeline and WED-trained mass head. — WED mass metrics documented; multi-item and transfer validation still limited.
- Nutrient values depend on recipe and food composition.: Component shares and explicit per-100 g nutrient table. — Several menu rows remain assumed and need recipe-specific ground truth.
- A point estimate hides uncertainty.: Monte Carlo distributions, intervals, coverage factors and probabilistic verdicts. — Intervals currently under-cover relative to nominal 90%; calibrated confidence claims are not supported.
- A hard binary label may overstate weak evidence.: Input gates, evidence coverage, BORDERLINE and cannot_verify outcomes. — Decision policy is implemented and tested at code/contract level; field validity needs further evaluation.
- Manual monitoring can be difficult to scale.: Browser UI, API and reusable menu/policy endpoints. — Local prototype built; operational production budgets and sustained-load testing are open.

3.1.7 Completed Activities Recorded in the Repository

- Modular backend stages and a structured analysis contract have been implemented.
- Core model integrations for SAM 2.1, SigLIP 2 and MoGe-2 are present in the inference pipeline.
- ArUco marker detection, EXIF parsing, camera pose estimation and a measured/prior anchor distinction are implemented.
- A WED-trained learned mass-estimation head and class mappings are included in the current repository.
- Monte Carlo nutrient sampling, multi-item aggregation and policy-based compliance assessment are implemented.
- A React/TypeScript frontend and Zod response-schema contract are included.
- Capture protocols, data manifests, evaluation reports, model-readiness documentation and automated tests have been added.

- The readiness record reports 70 passing frontend tests and a successful production frontend build in the recorded environment; current backend verification is qualified because the full run did not complete in the restricted 2026-10-30 environment.

3.2 Justification of the Selected Problem

The selected problem is appropriate for an AI and Machine Learning major project because it requires the integration of several distinct AI capabilities with domain-specific rules. Food segmentation is a dense visual prediction task; dish recognition is a semantic representation task; depth estimation and camera geometry are 3D inference tasks; portion estimation is a regression task; nutrient estimation is a propagation-of-uncertainty problem; and the final assessment is a constrained probabilistic decision problem. The end-to-end result cannot be solved reliably by a single classifier.

The problem also has a clear social application and measurable intermediate outputs. Physical mass can be compared with kitchen-scale ground truth, recognition can be evaluated against labelled images, segmentation can be evaluated using overlap/area metrics, nutrient calculations can be checked against recipe/composition references, and API behavior can be tested with contract and boundary cases. This allows the project to be evaluated as an engineering system rather than as a demonstration-only interface.

A key reason for selecting this problem is that it makes model limitations visible. A trustworthy tool should know when a reference marker is missing, when a dish is unsupported, when a model stage fails, when the food-composition value is assumed and when uncertainty is too large for a definitive assessment. NutriSense explicitly encodes these cases into its response contract and policy logic.

3.3 Project Contribution to Society and Sustainable Development Goals

NutriSense is intended to support the monitoring of meal quality in school nutrition programmes. It does not replace programme administrators, nutrition experts or direct weighing. Its potential contribution is to make selected meal observations more structured and quantifiable, provided that its estimation accuracy and data-collection procedures are validated in representative field conditions.

Table 3.2: Project contribution to the Sustainable Development Goals

- SDG 2: Zero Hunger: Target 2.2 - end all forms of malnutrition. — Estimate energy and protein adequacy at the plate level and expose where additional measurement or nutritional intervention may be needed.
- SDG 3: Good Health and Well-being: Target 3.4 - reduce premature mortality from non-communicable diseases. — Support better visibility of meal adequacy as one component of broader nutrition and health monitoring; no individual health outcome is predicted.
- SDG 4: Quality Education: Target 4.1 - equitable and quality primary/secondary education. — Support the quality of school meal provision, which is one of the enabling conditions associated with student well-being and participation.

The project also supports responsible innovation by documenting uncertainty, data provenance and privacy constraints. The capture manual instructs users to photograph trays and food only and avoid children's faces, uniforms, name tags or class boards. Student names

are not intended to be stored in the golden dataset metadata. Required permissions and consent practices at the collection site must be followed.

CHAPTER 4

SYSTEM REQUIREMENT SPECIFICATION

4.1 Functional Requirements

Table 4.1: Functional requirements

	Functional requirement	Priority
1	The system shall provide a browser interface for submitting a meal image.	High
2	The user shall provide menu day and student class band; serving style may also be supplied.	High
3	The API shall validate upload size, image decoding, day, band and digital-zoom constraints.	High
4	The system shall detect a supported ArUco marker and establish a measured scale when the marker is valid.	High
5	The system shall segment food regions and provide a structured failure/abstention when no usable mask is produced.	High
6	The system shall map a supported dish to a configured menu/gallery identity or report an unsupported result.	High
7	The system shall estimate portion mass through the applicable geometry or learned path.	High
8	The system shall calculate energy and protein estimates from configured component mappings and nutrient rows.	High
9	The system shall propagate uncertainty and calculate coverage/probability values.	High
0	The system shall return PASS, FAIL, BORDERLINE, cannot_verify or out_of_scope as appropriate.	High
1	The API shall expose health, menu, goals and analysis endpoints.	Mediun
2	The result shall include reasons, assumptions, model versions, policy metadata and degradation information.	High

4.2 Non-functional Requirements

Table 4.2: Non-functional requirements

- Reliability: Failures in pipeline stages should be converted into structured failures/abstentions rather than unhandled server errors.
- Performance: Model inference should use available GPU acceleration where possible; inference is serialised per backend process to control VRAM use.
- Scalability: The current architecture is suitable for a controlled local prototype; production multi-worker and multi-user scaling require target-specific validation.
- Usability: The frontend should guide image capture and display structured estimates, reasons and policy notes in a readable form.
- Maintainability: Menu, policy, nutrient and calibration parameters are stored in configuration files and loaded through central configuration functions.
- Interoperability: The frontend and backend communicate through a JSON API; the frontend uses TypeScript and a generated Zod contract.
- Security: CORS origins and upload size are configurable. Authentication and TLS must be placed in front of the API for non-local deployment.

- Privacy: Only food/tray images should be collected; identifiable student information should not be embedded in the golden dataset.
- Auditability: The response carries model versions, policy digest, assumptions, reasons and degradation tags; evaluation manifests preserve provenance.
- Robustness: Invalid captures, low-quality evidence and unsupported cases should lead to warnings or abstention rather than fabricated measurements.

4.3 Hardware Requirements

The repository's documented development setup recommends an NVIDIA GPU with CUDA for practical model inference, while allowing CPU execution at lower speed. The readiness record identifies an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM as the measured local benchmark target. It also records that this is a development machine rather than an agreed production deployment platform.

Table 4.3: Hardware requirements and evidence

- Processor: Modern 64-bit CPU capable of running Python, Node and image processing. — Multi-core CPU for API, data processing and frontend build tasks.
- Memory: Sufficient RAM to load Python dependencies and image/model tensors. — 16 GB or more is a practical development target; exact minimum is not specified by the repository.
- GPU: CPU path is available but slower. — CUDA-capable NVIDIA GPU; benchmark record uses RTX 4060 Laptop GPU, 8 GB VRAM.
- Storage: Space for source code, datasets, model weights and runtime caches. — Several gigabytes of model weights and data are required; README estimates first-run Hugging Face model downloads at approximately 3 GB.
- Camera: Phone or camera capable of capturing original JPEG with EXIF metadata. — Main 1x camera, no digital zoom, high resolution (minimum side target 1280 px) and stable capture.

4.4 Software Requirements

The software environment uses the Python backend and model-inference libraries, a Node.js-based frontend toolchain, structured YAML/JSON/NPZ configuration and data files,

and automated testing and quality tools. The implemented stack is summarised in Table 6.1. Exact dependency versions should be taken from the project manifests at the final submission commit.

- Backend runtime: Python 3.14, uv — Python runtime and reproducible dependency management.
- API: FastAPI, Uvicorn, AnyIO — HTTP API, asynchronous request handling and bounded threadpool inference.
- Computer vision: OpenCV, Pillow, NumPy — Image decoding, colour conversion, ArUco detection, mask operations and numerical processing.
- Deep learning: PyTorch, TorchVision, Transformers — Model execution and pretrained model integration.
- AI models: SAM 2.1 Hiera-Large, SigLIP 2 Base, MoGe-2, DINOv2-based weight head — Food segmentation, dish embeddings, monocular geometry/depth and learned mass regression.
- Data/configuration: PyYAML, JSON, NPZ, YAML configuration — Menu, standards, nutrient data, anchor configuration and gallery storage.
- Frontend: Node 20+, React 18, TypeScript, Vite — Interactive browser interface and build pipeline.
- Frontend validation: Zod, Vitest, Testing Library — Runtime contract validation and UI tests.
- Testing/quality: Pytest, Ruff, Pyright — Backend testing, linting and type-check baseline gate.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

NutriSense follows a layered client-server architecture. The browser frontend collects the image and contextual inputs, provides the user-facing workflow and renders the analysis contract. The FastAPI backend handles upload validation, route execution and inference orchestration. The engine layer coordinates model stages and decision logic. The model layer contains reusable model adapters and registries. Configuration and data files provide menu definitions, policy standards, food composition values, model mappings and evaluation parameters.

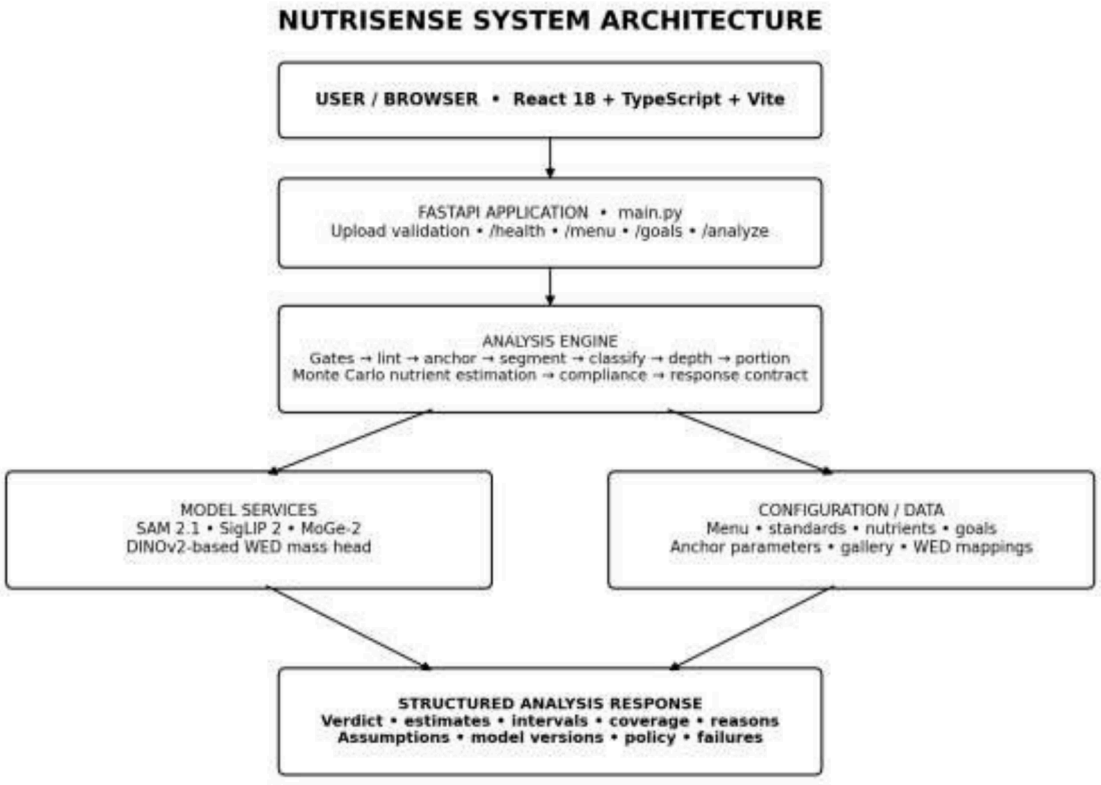


Figure 5.1: High-level system architecture

The backend loads expensive model assets lazily through a registry so that models need not be initialised at module import. A bounded AnyIO capacity limiter serialises inference to one request at a time per process, reducing GPU memory contention. The health and configuration endpoints remain asynchronous/lightweight and are not placed behind the same

inference limiter. A CUDA out-of-memory guard clears the model registry and retries a model stage once; a second failure is converted into a structured failure.

5.2 Methodology

The methodology is organised as a sequence of auditable stages. The pipeline's documented order is lint, anchor, segmentation, classification, depth, portion estimation, Monte Carlo sampling and verdict evaluation. Input gates and scope gates are applied at defined points. A fatal stage failure is returned as a structured failure; the anchor stage can degrade to a prior tier and continue with reduced evidence coverage.

Table 5.1: Main analysis pipeline stages and responsibilities

Stage	Main operation	Output / control
Input validation	Check upload size, image decoding, EXIF, day, class band and digital zoom.	Valid image/context or structured rejection.
Capture lint and scale anchor	Check resolution/marker adequacy; detect the known-size ArUco marker.	Measured scale when valid; otherwise an explicitly labelled prior tier.
Food segmentation	Generate candidate food masks with SAM 2.1; filter candidates and support multi-vessel processing.	Food mask(s) and segmentation diagnostics.
Dish recognition	Embed the food crop with SigLIP 2 and compare against the configured gallery.	Supported dish identity or scope abstention.
Depth and portion estimation	Use MoGe-2 geometry and scale calibration; use eligible WED learned mass head where configured.	Estimated portion mass, method and uncertainty.
Nutrient estimation	Combine component masses with configured composition values and Monte Carlo sampling.	Energy/protein distributions and intervals.
Compliance assessment	Evaluate nutrient threshold probabilities together with evidence coverage.	PASS / FAIL / BORDERLINE or abstention.
Response and cleanup	Serialize the analysis contract and remove temporary upload data.	Auditable response with reasons, assumptions and provenance.

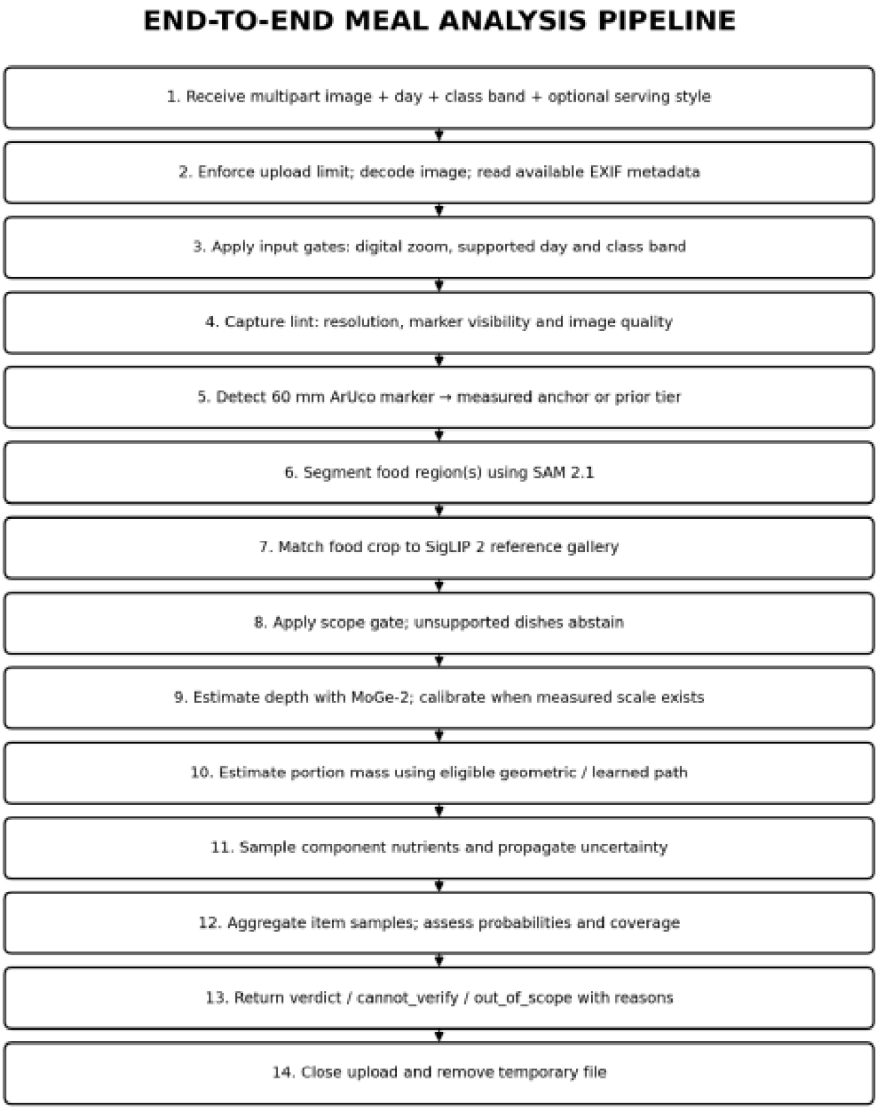


Figure 5.2: End-to-end analysis pipeline

5.2.1 Image Capture and Input Validation

The analysis endpoint accepts a multipart image, menu day, student class band and optional serving style. The request body is written to a temporary file in 1 MiB chunks. The default maximum upload is 12 MiB, configurable using NUTRISENSE_MAX_UPLOAD_BYTES. The endpoint decodes the image using OpenCV, reads EXIF information, and passes the decoded BGR image and metadata to the pipeline. The temporary file is deleted in a finally block and the upload stream is closed regardless of success or failure.

The pipeline applies gates for digital zoom, supported day and valid class band. Digital zoom other than 1.0 is rejected because digital zoom breaks the assumed image geometry/capture

protocol. The menu file is the single source of truth for available dishes and days. The class-band vocabulary is restricted to 1-5, 6-8 and 9-10.

5.2.2 Metric Scale Anchor Using ArUco

The reference marker is a 60.0 mm ArUco marker (DICT_4X4_50, marker ID 7) printed at 100% scale. The system detects marker corners and estimates average side length in pixels. The image scale is calculated as: [5].

cm_per_pixel = marker_size_cm / marker_side_pixels

The physical marker dimension is 6.0 cm. Camera intrinsics are estimated using EXIF focal length, with a preference for 35 mm-equivalent focal length. Marker pose is recovered using camera geometry; a reprojection error above the configured tolerance rejects the pose while retaining the two-dimensional scale if valid. The implementation includes a quiet-zone test to reject false detections around dark borders. If a usable marker is absent or too small, the anchor label becomes prior and the result must carry a statement that the scale/weight is estimated.

5.2.3 Food Segmentation

SAM 2.1 is queried with a central positive point prompt. It returns multiple mask candidates. NutriSense computes mask area fraction and solidity and checks whether candidate masks are nested. This is motivated by the common relationship in food photographs where the food mask is contained inside a larger plate or scene mask. A selected mask should correspond to the food rather than the entire plate. If the central point produces an unusable speck or an excessively broad mask, a box-prompted query is attempted. The segmentation module returns a binary mask and diagnostic metadata such as strategy, model score, candidate count and area fraction.

For multi-vessel images, nine image locations are queried within a coarse central box. The resulting masks are deduplicated by IoU, sorted by area and truncated at four items. The multi-item route is included in the implementation but requires more field validation, as described in the readiness status.

5.2.4 Dish Classification

The segmented/cropped food region is passed to the SigLIP 2 encoder. The system compares its embedding with a stored dish gallery. The chosen gallery identity is mapped to menu and

serving definitions. A gallery build tool produces the stored gallery from class reference images. The repository notes a current gallery size of 46 images and an internal leave-one-out result of 46/46, with an abstention threshold of 0.82. That result characterises the gallery check and must not be interpreted as a measured real-world canteen recognition rate.

5.2.5 Depth Estimation and Portion Mass

MoGe-2 provides monocular depth/geometry. When a measured marker anchor is present, the predicted depth scale is calibrated against the known marker geometry. The calibrated depth and food mask support the estimation of surface footprint, food height and volume. The portion estimator uses dish-specific density and composition information to convert estimated volume to mass. A simplified physical relationship is:

estimated_mass_g = estimated_volume_ml x effective_density_g_per_ml

When the reference marker is unavailable, the project distinguishes between size-prior geometry and a learned weight-head path. The WED-trained mass head predicts mass from the image and segmentation mask using mapped class slots. Its result is tagged as learned_weight and its uncertainty is taken from class-specific validation residuals where available, otherwise a fallback relative sigma is used. The system does not claim that such a learned estimate is a measured scale. No-card cases are explicitly labelled as estimates.

5.2.6 Nutrient Composition and Monte Carlo Estimation

The menu configuration defines dish components, generic nutrient identifiers, nominal recipe shares and effective density values. The food composition table expresses energy and protein per 100 g edible portion. When a component has estimated mass m_i, energy density e_i and protein density p_i, point estimates are:

Energy_kcal = sum_i (m_i / 100) x e_i Protein_g = sum_i (m_i / 100) x p_i

The Monte Carlo stage represents uncertain quantities using repeated samples. In the current pipeline, 4,000 samples are used by default unless overridden. The sampler propagates portion mass and composition uncertainty to energy and protein distributions. For multi-item meals, per-item sample arrays are summed elementwise so that aggregate meal intervals are computed from the combined sample distribution rather than by simply adding interval

endpoints. The output includes means and 90% intervals, but the readiness report explicitly states that observed interval coverage is below nominal 90%; these intervals must therefore not be described as fully calibrated confidence intervals.

5.2.7 Nutritional Adequacy Assessment

The project's central configured minimums are 450 kcal and 12 g protein for classes 1-5, and 700 kcal and 20 g protein for classes 6-8. The 9-10 band uses a 700 kcal/20 g mapping inherited from the upper-primary reference in the application. The standards file explicitly records that a separate statewide Karnataka class IX-X energy/protein minimum is not established by its cited sources. Accordingly, the 9-10 values should be described as the application's mapping, not as an independently published state norm.

Table 5.2: Nutritional compliance thresholds encoded in NutriSense

Class band	Minimum energy	Minimum protein	Evidence note
1-5	450 kcal	12 g	Central PM POSHAN minimum as encoded in standards.yaml.
6-8	700 kcal	20 g	Central PM POSHAN minimum as encoded in standards.yaml.
9-10	700 kcal	20 g	Application mapping inherited from 6-8; not stated as a separate statewide minimum.

For each mandatory nutrient, the engine calculates the fraction of Monte Carlo samples at or above the configured minimum. Let P_n denote that probability and C denote the evidence coverage score. NutriSense applies the following decision rules:

PASS: all mandatory nutrients have P_n >= 0.90 AND C >= 0.85 FAIL: at least one mandatory nutrient has P_n <= 0.10 AND C >= 0.90 BORDERLINE: all other admitted cases

Coverage is a multiplicative score derived from evidence degradation factors, including prior anchor, estimated weight, table-prior base, degraded quality and uncalibrated depth. The prior geometry path is deliberately capped and cannot issue a definitive PASS/FAIL. The learned weight path is also marked estimated and receives an estimated-weight coverage factor. Salt is advisory only and is not a verdict nutrient. Raw ingredient allocations are diagnostic only; they must not be directly compared to cooked plate mass because cooking yield changes mass.

5.3 Detailed Design

5.3.1 Class Diagram

The source describes NutriSense as a service-oriented implementation rather than a single object-oriented class hierarchy. Figure 5.3(a) is therefore a conceptual class/module view: it maps the application entry point, configuration loader, pipeline stages and response records. It does not imply that every box is a Python class.

FIGURE 5.3(a): CONCEPTUAL CLASS / MODULE DIAGRAM

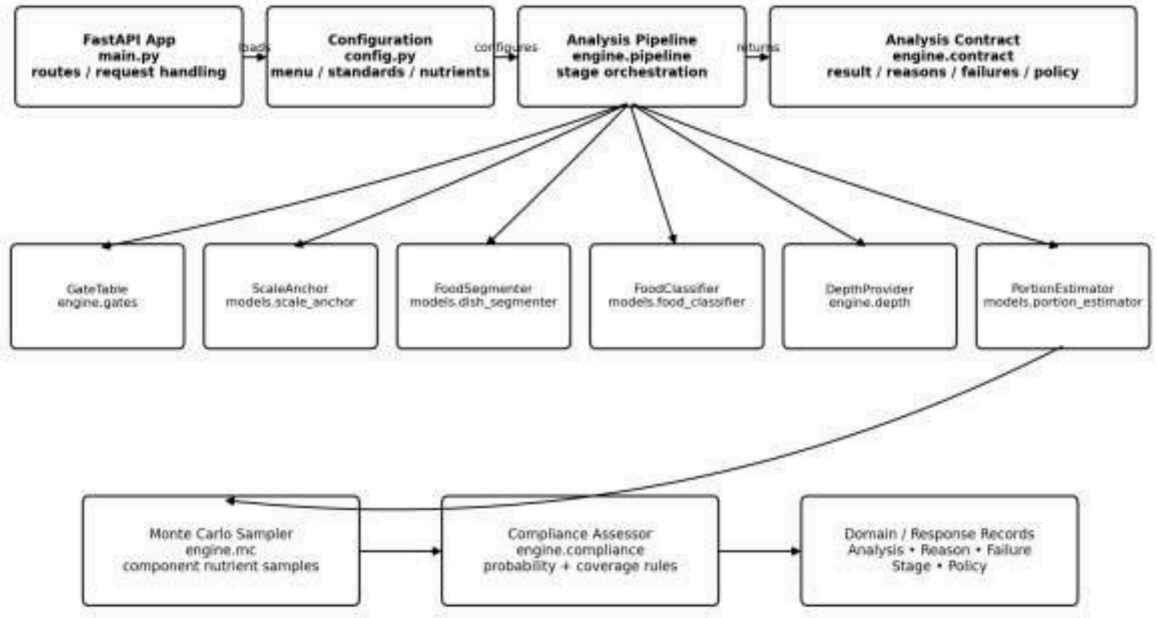


Figure 5.3(a): Conceptual class and module diagram

The main conceptual entities are Analysis, Reason, Failure, Stage, Policy and the model-stage outputs. Analysis holds the final verdict and component outputs. Reason communicates why a verdict or abstention occurred. Failure records a failed stage with its category and details. Stage identifies the processing stage associated with a failure. Policy captures the thresholds and policy-version digest used to evaluate the sample. This separation supports consistent error reporting and prevents frontend code from independently re-deriving policy decisions.

5.3.2 Activity Diagram

The activity diagram models the request path and its documented decision branches: file and context validation, measured-versus-prior anchor handling, supported-dish checks, structured abstention and response cleanup.

FIGURE 5.3(b): ACTIVITY DIAGRAM

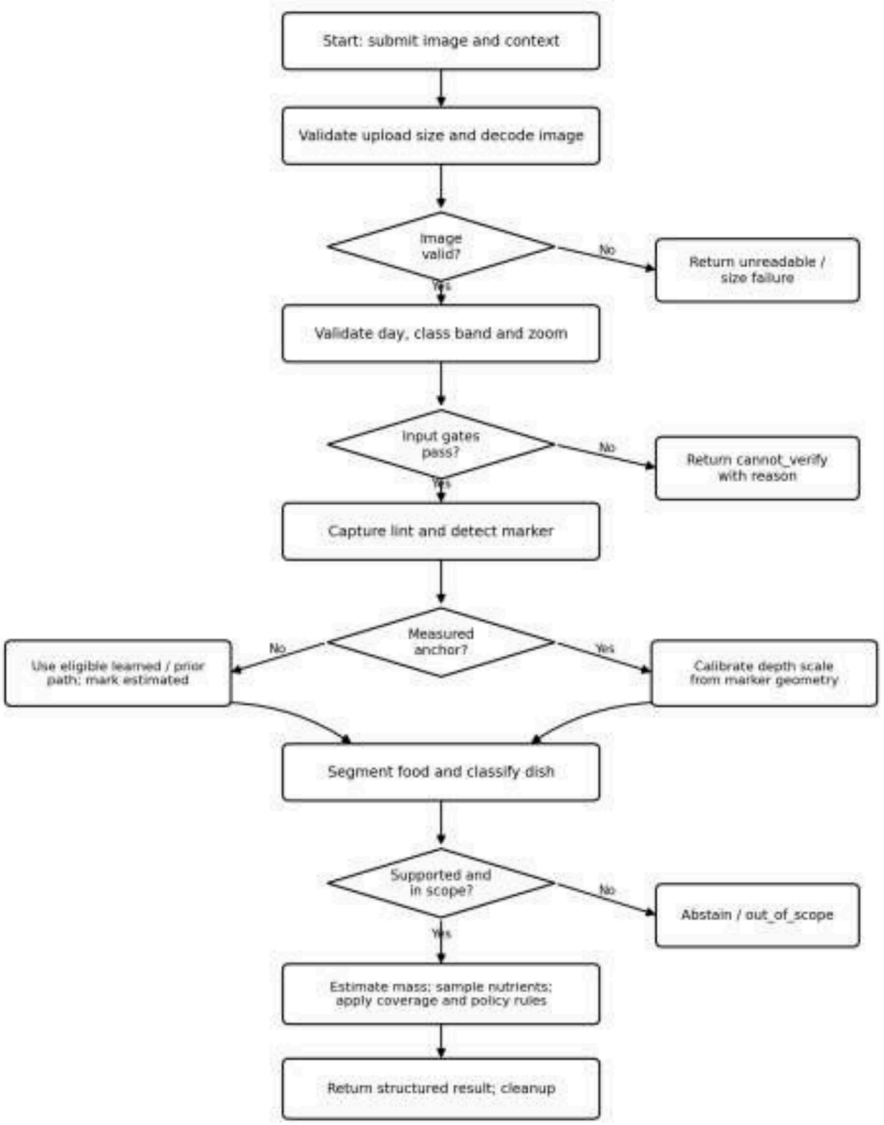


Figure 5.3(b): Activity diagram for meal analysis

5.3.3 Use Case Diagram

The meal-monitoring user is the primary application actor. The user selects the relevant context, submits a meal image and reviews the returned estimate and evidence. The administrator/developer shown is a configuration maintainer; the diagram does not assert the presence of a separate in-app administration console.

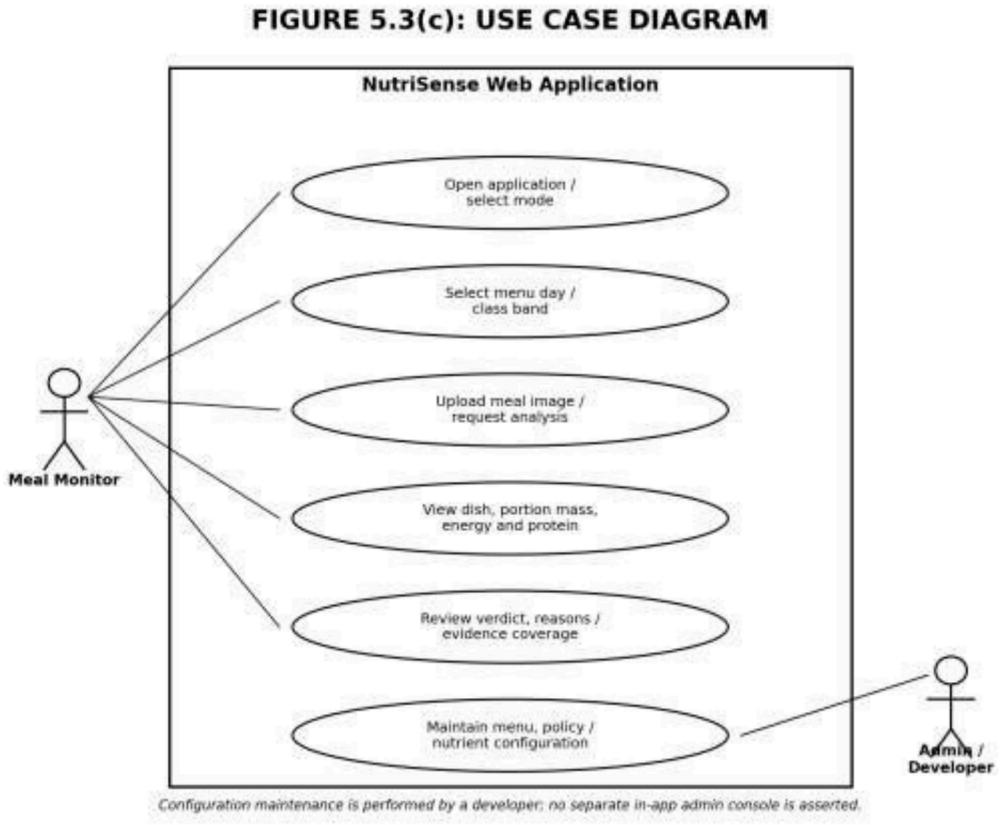


Figure 5.3(c): Use case diagram

Table 5.3: Analysis response contract (selected fields)

Field group	Purpose in the response
verdict / compliance	Carries the policy outcome and nutrient-level probability information.
reasons / failures	Explains gate rejections, abstentions and stage failures.
lint / anchor	Records capture-quality checks and the scale-anchor method.
segmentation / classification / dish	Describes food-mask and dish-identification outputs.
portion	Reports estimated mass, the selected estimation path and uncertainty.
nutrition	Reports energy/protein estimates and their sampled intervals.
coverage / assumptions	Communicates evidence quality, priors and modelling assumptions.
model_versions / policy	Records model and policy/configuration provenance.
path_used / degradations / items	Describes the processing route, reduced-evidence conditions and item-level outputs.

CHAPTER 6

IMPLEMENTATION

6.1 Overview of Tools and Technologies Used

The project uses Python for model orchestration and API services and TypeScript/React for the interactive browser client. FastAPI exposes HTTP endpoints and Pydantic-backed data structures through the backend contract. PyTorch provides tensor operations and model execution. OpenCV is used for image decoding, colour conversions, ArUco detection and geometric processing. The Transformers library loads pretrained SAM 2.1 and SigLIP 2 components. MoGe-2 supplies monocular geometry. The configuration layer uses YAML, JSON and NumPy NPZ files to keep policy, menu and gallery information structured and versionable.

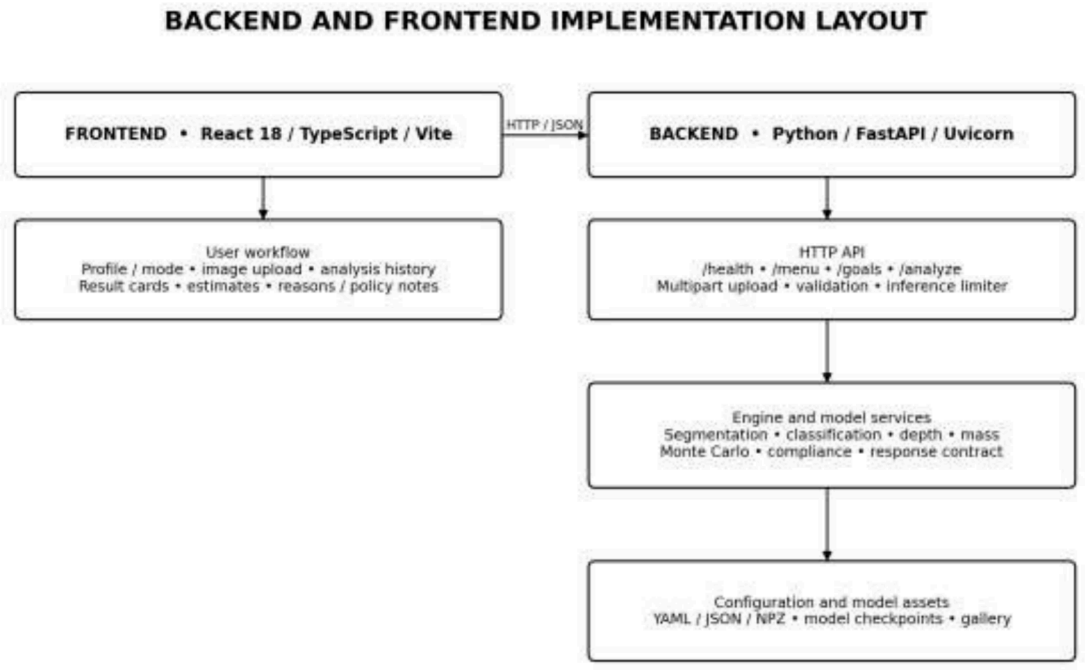


Figure 6.1: Backend and frontend implementation layout

Table 6.1: Software and technology stack

Technology	Role in NutriSense	Repository evidence
Python 3.14	Core backend language and inference orchestration.	pyproject.toml and backend source files.
FastAPI + Uvicorn	REST API, multipart upload and local server.	main.py exposes /, /health, /menu, /goals and /analyze.
PyTorch + Transformers	Deep learning runtime and pretrained model loading.	Model adapters in models/ and engine/depth.py.
SAM 2.1	Food segmentation and vessel-mask candidates.	models/dish_segmenter.py.
SigLIP 2	Gallery-based visual embedding/classification.	models/food_classifier.py; README model versions.
MoGe-2	Monocular depth/geometry.	engine/depth.py; README model versions.
DINOv2-based head	Learned food mass regression for mapped no-card cases.	models/weight_head.py and data/weight_head.pt when present.
OpenCV + Pillow	Image handling, ArUco detection, camera metadata.	main.py and models/scale_anchor.py.
NumPy + PyYAML	Numerical estimation, sampling and configuration loading.	engine/ modules and config.py.
React 18 + TypeScript + Vite	Frontend workflow and result visualisation.	frontend/package.json and frontend/src/.
Zod + Vitest	API response validation and frontend tests.	Generated contract type and frontend test suite.
Pytest + Ruff + Pyright	Backend test, lint and type-check gates.	pytest.ini, ruff.toml, pyrightconfig.json and CI workflow.

6.2 Details of the Modules

6.2.1 API and Request Handling Module

The entry point main.py creates the FastAPI application, configures CORS, logging and the inference limiter, and exposes the API routes. The / endpoint provides a basic service message. /health provides a health state. /menu returns configured dishes, days, class bands, capture fields and policy settings. /goals returns general WHO daily-goal reference data by age band, sex and activity level; the source code documents that this read-only general-mode endpoint does not alter school-meal compliance. /analyze accepts the multipart form and invokes the inference pipeline in a bounded worker thread.

6.2.2 Configuration and Data Management Module

config.py centralises file paths and YAML/JSON/NPZ loading. It uses repository-relative paths rather than the process working directory, and caches parsed configuration objects. The module includes loaders for menu, standards, nutrients, yields, general goals, ingredient densities, anchor configuration, gallery embeddings and WED dish mappings. Centralised

loading avoids each subsystem maintaining a separate interpretation of menu or policy values.

6.2.3 Input Gates and Capture Lint Module

engine.gates defines a single gate table. The input gates evaluate digital zoom, supported menu day and class band. A separate scope gate is evaluated after the dish is classified. The

capture lint checks technical conditions such as image resolution and marker adequacy. Gate rejection returns a failure and a reason through a common helper, ensuring that the failure record and displayed reason derive from the same source.

6.2.4 Scale Anchor and Camera Geometry Module

models.scale_anchor reads relevant EXIF values, detects the configured ArUco marker, validates its quiet zone and estimates centimetres per pixel from its known size. It also attempts to estimate marker pose and camera distance using a camera matrix. The anchor result includes a method label, scale interval, homography/pose information, tilt, distance and diagnostic extras. When marker detection fails, the module returns a prior anchor rather than a false metric measurement.

6.2.5 Segmentation and Classification Modules

models.dish_segmenter wraps SAM 2.1 loading and inference. A model registry provides lazy initialisation. The segmentation result contains a binary mask and diagnostic fields. The same module contains a multi-vessel item segmentation routine using grid prompts and IoU deduplication. models.food_classifier wraps SigLIP 2 and gallery lookup to map the food region to a dish identity. The classifier and gallery vocabulary are related to menu configuration and WED mappings.

6.2.6 Depth and Portion Estimation Modules

engine.depth wraps the MoGe-2 model and scale-calibration procedure. models.portion_estimator combines segmentation, depth/geometry, dish component definitions and density assumptions to estimate mass, volume and uncertainty. The engine chooses between the geometric route and the learned WED mass route according to anchor tier and dish mapping. The response indicates which portion path generated the grams, such as card_geometry, ingredient_head, learned_head, prior_geometry or multi_item; null is used when no portion result was produced.

6.2.7 Nutrition Monte Carlo and Compliance Modules

engine.mc samples component nutrient values and aggregates item-level samples into a meal-level distribution. It stores nutrient composition provenance, including the source table hash, component grams, densities and point contributions where available. engine.compliance computes nutrient probabilities and evidence coverage, then selects PASS, FAIL or

BORDERLINE according to the standards configuration. Salt remains advisory; raw ingredient allocation figures are diagnostics, not verdict inputs.

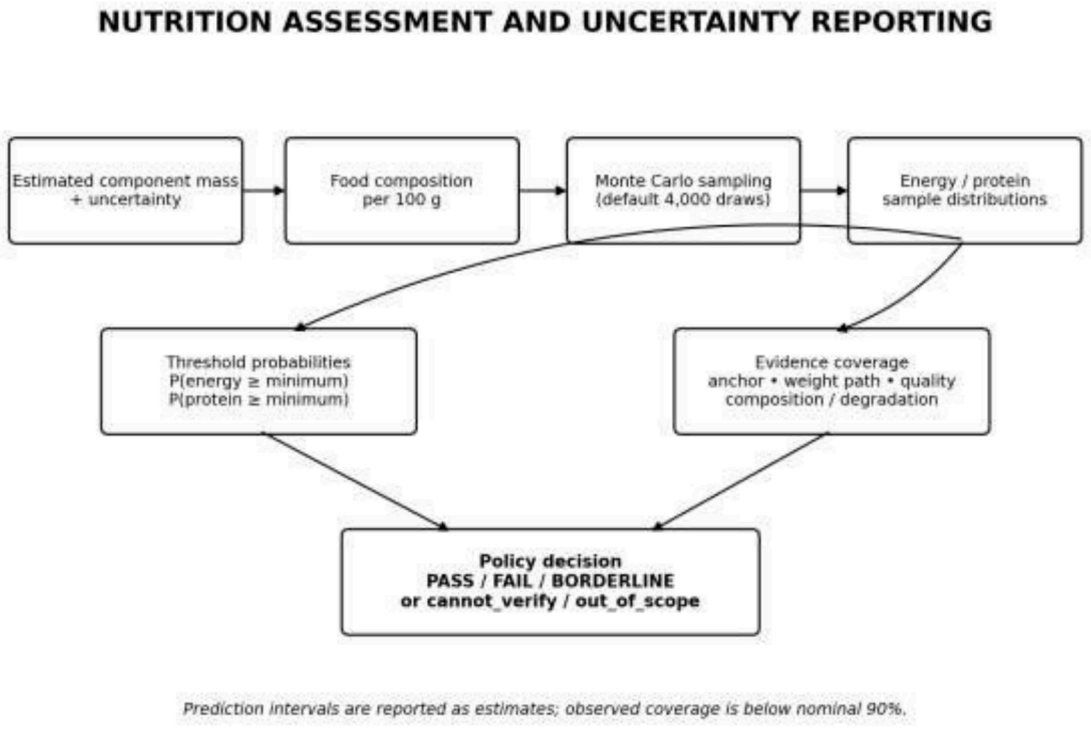


Figure 6.2: Nutrition assessment and uncertainty reporting

6.2.8 Analysis Contract and Frontend Module

engine.contract owns the structured response types and helper functions for stage failures, gate rejections, policy loading and logging. The frontend consumes an API contract represented in TypeScript and Zod. The README describes a 19-key response contract with fields including verdict, reasons, failures, lint, anchor, segmentation, classification, dish, portion, nutrition, coverage, assumptions, advisory, compliance, model_versions, policy, path_used, degradations and items. The exact wire version should be taken from the current generated contract file at the final submission commit; the repository notes that contract evolution is managed centrally to preserve compatibility.

6.3 Challenges Encountered and Strategies Used to Address Them

Table 6.2: Implementation challenges and mitigation strategies

Challenge	Effect on the system	Mitigation strategy documented in project
Food mask includes plate/rim/table context.	Area and depth may be overstated, causing large mass error.	SAM candidate solidity/area/containment selection and box-prompt fallback; further real-photo validation required.

Challenge	Effect on the system	Mitigation strategy documented in project
Small/degenerated centre-point masks.	A tiny mask may look valid to downstream stages but represent only a fragment.	Treat masks below area threshold as failed and retry using a box prompt.
Missing or poor-quality scale marker.	No defensible physical image scale.	Measured/prior anchor tiers, marker capture protocol and explicit estimate disclaimer.
Monocular depth scale ambiguity.	Depth output may be relative or inaccurate in metric units.	Calibrate depth using measured marker geometry; reduce coverage when calibration is not available.
GPU memory exhaustion.	Model inference may fail during loading or execution.	Clear model registry/cache and retry once; report structured failure if retry fails.
Model loading time and memory.	Large pretrained models increase startup cost and VRAM pressure.	Lazy model loading, process-level serial inference and CUDA/CPU device selection.
Variable dish recipes and uncertain composition.	Energy/protein estimates can be biased even if mass is correct.	Store explicit assumptions and nutrient provenance; use wider ranges for generic recipes.
Multi-vessel segmentation and assignment.	The same component may be assigned to multiple vessels or masks may spill.	Separate vessel masks and meal aggregation are implemented; larger, independently weighed datasets remain necessary.
Uncalibrated uncertainty intervals.	Nominal 90% intervals may not contain 90% of true values.	Calibration analyses performed; current calibration candidate not promoted because it fails the pre-registered width/coverage gate.
Runtime/testing environment variability.	Some backend tests stalled in restricted CPU/sandbox execution.	Record verification status per environment; avoid claiming an unreplicated complete suite pass.

CHAPTER 7

TESTING AND RESULTS

Testing is considered at three levels: unit testing of individual functions and modules, integration testing of the API and pipeline boundaries, and system-level evaluation of frontend/backend behaviour and model quality. The repository contains test suites and documented evaluation records. This report distinguishes collected test counts, passed test results, benchmark findings and model-quality metrics. A collected test count does not establish that the tests passed.

7.1 TESTING

7.1.1 Unit Testing

Unit tests verify individual functions and module contracts. The codebase includes tests for segmentation selection, marker/anchor behaviour, pipeline gates, portion estimation, Monte Carlo calculations, compliance logic, failure contracts, weight-head loading and evaluation metrics. Unit-level cases are used to verify both nominal behaviour and edge cases such as malformed images, absent marker evidence, unsupported inputs and failed model stages.

The model-readiness record reports focused test results for weight-head loading, evaluation metrics, grouped cross-validation split isolation, Nutrition5k nutrient evaluation and nutrition provenance. It also records a lock-guard validation of 502 checks with zero skipped and zero failed in the current verification attempt. These focused outcomes should be cited separately from the full backend suite.

7.1.2 Integration Testing

Integration testing checks whether the API routes, configuration loaders, model registry, inference pipeline, response contract and frontend expectations work together. The API has tests for malformed and oversized uploads and the per-process inference limiter. A happy-wire contract fixture check was recorded as passed. The frontend contains tests for wire parity, selectors, wizard interactions and policy-note rendering.

The readiness document states that the frontend Vitest suite passed 70 tests across 11 files and that the production frontend build completed. It also records a backend verification limitation

on 30 October 2026: 281 tests were collected (271 non-GPU and 10 GPU-marked), but the full current run did not complete in the restricted environment. An earlier local

verification record reports 281 passed, but the later run did not reproduce that result. The report therefore does not claim that the current full backend suite passed in the latest verification environment.

7.1.3 System Testing

System testing considers the complete path from a browser upload to a structured analysis response. The README documents the local backend on port 8731 and frontend on port 5173. The API includes GET /health, GET /menu, GET /goals and POST /analyze. The /analyze route enforces a default 12 MiB upload cap, reads the image and EXIF data, executes inference through a one-slot limiter and removes the temporary upload in a finally block.

The readiness record includes API benchmarks from an RTX 4060 Laptop GPU development machine. Two sequential real-canteen-photo scored calls had a reported p50 of 8.263 seconds and p95 of 8.338 seconds, with all responses returning HTTP 200 and a FAIL verdict. The sample size of two calls is too small to establish a stable latency percentile. Separate eight-call tests produced p95 values of 735 ms and 751 ms, but all returned cannot_verify; these numbers represent fast abstention paths, not successful analysis. Sustained load, cold-start distributions, remote network behaviour and multi-worker concurrency remain unmeasured.

7.2 RESULTS

7.2.1 Portion-Mass Estimation Results on WED

The repository’s current deployable WED head is documented with a DEV baseline of 15.034 g MAE across 108 views, 21 plates and 1,218 ROIs. A plate-bootstrap 95% confidence interval for the mean MAE is recorded as 12.227-18.889 g. The one-time WED LOCKED evaluation reports 14.667 g MAE across 2,793 ROIs and 48 plates, with a plate-bootstrap 95% interval of 13.331-16.029 g. These metrics refer to mass regression on the WED evaluation setup and not to an end-to-end measurement of energy or protein accuracy on school meals.

Five-fold grouped cross-validation over WED TRAIN plates is reported at 16.421 g pooled MAE across 11,759 ROIs and 198 held-out plates. The fold MAEs are 15.684, 15.720, 16.184, 16.521 and 18.096 g, with fold standard deviation 0.884 g. The repository describes this as a TRAIN-only grouped generalisation estimate and not as a comparison against the LOCKED test set.

7.2.2 Uncertainty Calibration

The nominal 90% prediction intervals are documented as under-covering. The readiness report states 85.22% coverage on the measured DEV carve and 82.77% on the one-time LOCKED evaluation. A global calibration multiplier required to reach 90% coverage increased interval width by 47%, exceeding the pre-registered 25% width cap. Per-class calibration also failed the recorded coverage and width gates. Therefore, the project must not present the present intervals as calibrated 90% confidence intervals.

7.2.3 Nutrition Estimation and Validation

The nutrition engine calculates energy and protein by multiplying estimated component grams with per-100 g composition values. The nutrient table identifies some rows as literature-derived and others as assumed. In particular, sambar, vegetable rice and Bisi Bele Bath rows are marked assumed. The menu configuration also uses assumed component shares and bulk densities for some dishes. These assumptions are carried into the analysis result rather than silently treated as measured truth.

The readiness record reports an exploratory Nutrition5k held-out evaluation over 47 dish-ID-held-out dishes. The unfiltered all-slot output has very high errors (3,208.595 g total-mass MAE, 2,916.463 kcal MAE and 126.724 g protein MAE among the documented outputs) and is not considered a usable meal prediction. An oracle-present-ingredient diagnostic gives lower errors, but it uses held-out ingredient labels and is explicitly non-deployable. Neither result supports a serving-quality nutrient accuracy claim. A validated food-presence/identity and portion model with compatible ground truth is still required.

7.2.4 Real-capture Pilot and Multi-item Findings

The 30 October 2026 real-capture pilot covered 23 angle views of six unchanged plates. Twenty views had measured anchors and three had prior anchors. The repository records unusually large measured multi-item mask-area sums relative to the top-level food mask (2.84 to 7.71 times on multi-item records). A 14-pair same-image comparison showed

different MAE behaviour between card and forced-prior paths; however, the repository warns that the small pilot does not establish generalisation and that the prior result may reflect error cancellation. These plates are permanently labelled evaluation-only and must not be reused for model tuning.

Table 7.1: Project evaluation metrics and documented evidence

- WED DEV weight baseline: 15.034 g MAE; 108 views; 21 plates; 1,218 ROIs. — Development estimate; same DEV carve used in model/epoch selection.
- WED LOCKED weight evaluation: 14.667 g MAE; 2,793 ROIs; 48 plates. — One-time locked mass evaluation; must not be used for tuning.
- WED TRAIN grouped 5-fold CV: 16.421 g pooled MAE; 11,759 ROIs; 198 plates. — Grouped TRAIN-only generalisation estimate.
- 90% interval coverage: 85.22% DEV; 82.77% LOCKED. — Under nominal 90%; calibration success criterion not met.
- Frontend testing: 70 tests passed across 11 files; build succeeded. — Recorded in production-readiness file.
- Backend verification on 2026-10-30: 281 tests collected; current full run did not complete. — Do not report latest full-suite pass.
- Real-capture pilot: 23 views across six plates. — Exploratory, evaluation-only; not a production accuracy claim.
- API benchmark scored path: Two sequential calls; p50 8.263 s, p95 8.338 s. — Insufficient sample for stable percentile; local RTX 4060 only.
- Nutrition5k held-out exploration: 47 dishes; full-head output unusable; oracle-present diagnostic non-deployable. — No validated end-to-end nutrient accuracy claim.

7.2.5 Overall Readiness Assessment

The project readiness record identifies a reproducible frozen baseline and dataset provenance audits as met. The model-improvement criterion is not met because the tested candidate failed a pre-registered subgroup guardrail. Uncertainty calibration is not met for confidence claims. Production nutrition quality is not met because the Nutrition5k candidate is rejected and complete meal-level nutrition ground truth remains inadequate. Operational readiness

remains open because deployment hardware and numeric latency, memory, timeout, concurrency and error-rate budgets have not been agreed.

The correct conclusion from the results is that NutriSense has a substantial, modular research implementation and documented intermediate mass-estimation evidence, but additional field validation and model improvement are required before it should be relied on for school-meal serving decisions. The system is useful as a prototype for structured data collection, model experimentation and evidence-aware reporting.

CHAPTER 8

PROJECT MANAGEMENT

8.1 Gantt Chart

The repository identifies the team and the technical workstreams, but it does not provide a final approved calendar with actual start/end dates for every activity. The following is therefore a work-breakdown view for report presentation and should be replaced or adjusted to match the team's actual project log before submission.

Table 8.1: Indicative project work plan / Gantt chart

- Problem definition and literature survey: ■ — ■
- Data and policy research: ■ — ■ — ■
- System architecture and API design: ■ — ■
- Model integration: segmentation, recognition, depth: ■ — ■ — ■
- Portion and nutrient estimation: ■ — ■
- Frontend implementation: ■ — ■
- Integration, testing and evaluation: ■ — ■
- Documentation and final review: ■ — ■

Phase labels are indicative and not dates. Enter the actual weeks/months and project milestones agreed by the guide before final submission.

8.2 Team Roles and Responsibilities in Project Execution

Table 8.2: Team roles and responsibilities

- Aryan Kumar: 1BM23AI037 — Literature survey, research and data collection.
- Roshanth V: 1BM23AI155 — Literature survey, research and data collection.
- S S Gokula Swamy: 1BM23AI158 — Backend and frontend development.
- Somanath S D: 1BM23AI187 — Backend and frontend development.

These responsibility areas are taken from the NutriSense repository README. The final report should also include each member's specific deliverables and review/verification contributions if the team maintains a more detailed responsibility log.

8.3 Challenges Encountered and Mitigation Strategies

The project combines research experimentation, data collection, model integration and software engineering. The most significant coordination risks are dependencies between stages, shared configuration changes, large model downloads and GPU memory requirements, and the need to separate training/development data from evaluation-only data. These risks have been addressed through modular ownership, shared response contracts, configuration centralisation, model registry control, pinned dependencies and dataset provenance manifests.

Data evaluation discipline is especially important. The project uses a pilot/dev/holdout strategy for the golden set and a grouped split discipline for physical plates/views. The capture documentation states that holdout data should remain sealed until the final evaluation and must not be used for threshold or model tuning. The real-capture ArUco pilot is also marked evaluation-only. These rules reduce the risk of leakage and optimistic performance claims.

The remaining project-management tasks include expanding independent field captures, agreeing measurable production service-level objectives, reviewing the model readiness gates, validating the recipe composition table and obtaining the guide's approval of the final report and research-paper submission plan.

CONCLUSION & FUTURE WORK

NutriSense demonstrates an integrated AI/ML workflow for estimating food portion mass and assessing selected nutritional adequacy indicators from meal images. The project combines image segmentation, visual embedding-based dish recognition, fiducial marker geometry, monocular depth estimation, learned mass regression, Monte Carlo uncertainty propagation and probability-based compliance rules. Its principal engineering contribution is the separation of model inference, evidence quality, nutrition calculation and policy assessment into inspectable stages with a structured result contract.

The implementation includes a React/TypeScript interface, FastAPI backend, centralised configuration system, model registry, data/policy endpoints, automated testing and evaluation documentation. The repository records WED mass-estimation results and a one-time locked evaluation. It also records important limitations: the current nominal 90% intervals under-cover, multi-item segmentation remains unstable on some real images, several nutrient rows and recipe parameters are assumed, nutrition model transfer has not passed its readiness gate, and production operational targets have not been established.

Therefore, the project should be regarded as a research prototype and decision-support system. It should not be used as the sole basis for nutrition programme compliance decisions until representative field validation, improved calibration and compatible nutrient ground truth are available. In its current state, NutriSense provides a useful platform for reproducible experiments and structured observations, while explicitly communicating when the image evidence is inadequate.

Future Work

The project can be further improved by expanding the real-world dataset to include hundreds of independently weighed food plates collected across multiple cooking sessions, camera angles, serving vessels, food textures and portion-size categories. Development and holdout datasets should be separated based on physical plates and cooking sessions to ensure reliable evaluation. Segmentation accuracy can be enhanced, particularly for bowls, reflective plates, mixed foods and multi-vessel trays, by collecting pixel-level annotations or validated masks for a representative real-world photo benchmark.

Dish recognition can be improved by developing a larger and more diverse food image gallery and evaluating it using field-based data splits. Performance should be measured using precision, recall, confusion matrices and abstention coverage. Similarly, mass estimation should be strengthened using diverse training and validation datasets, class-specific error analysis and predefined evaluation criteria while preserving a separate locked evaluation set. Ingredient composition and food density values should also be validated against locally measured recipes and authoritative food-composition tables, replacing assumed values only when reliable evidence is available.

Furthermore, a complete meal-level nutrition estimation model should be developed and validated using compatible ground-truth energy and protein measurements before reporting calorie or protein estimation errors. Uncertainty intervals should be recalibrated using group-separated datasets to ensure reliable coverage and appropriate interval widths. For deployment, suitable hardware, memory requirements, response latency, timeout limits, error rates and concurrent request targets must be established and tested under different workloads.

Finally, security measures such as authentication, HTTPS/TLS, secure data storage, retention policies and access logging should be implemented before deployment beyond a trusted local environment. Usability testing should also be conducted with intended field users, and a proper human-review process should be established to ensure that automated nutrition estimates remain advisory, transparent and auditable.

LEARNING OUTCOMES

The project provided practical learning in the design and implementation of an end-to-end AI/ML system. The main outcomes are summarised below.

- Understanding the distinction between an image-classification demo and a multi-stage computer-vision system in which segmentation, identity, geometry, mass and nutrition estimation affect one another.
- Practical exposure to pretrained vision models, including SAM 2.1 segmentation, SigLIP 2 visual embeddings, MoGe-2 monocular geometry and DINOv2-based feature extraction for mass regression.
- Understanding camera calibration, EXIF metadata, ArUco fiducial markers, homography, camera pose and the limitations of physical scale recovery from a single image.
- Experience in designing data capture protocols with controlled camera settings, marker placement, weighing procedures, metadata recording, quality assurance and leakage prevention.
- Understanding probabilistic estimation and uncertainty propagation through Monte Carlo sampling, including the difference between nominal interval level and empirical interval coverage.
- Learning to encode policy rules as configurable, testable logic rather than embedding thresholds throughout model code.
- Backend engineering experience using FastAPI, asynchronous request handling, upload-size limits, inference concurrency controls, temporary-file cleanup, structured logging and typed API contracts.
- Frontend engineering experience with React, TypeScript, Vite, Zod validation and component-level automated testing.
- Learning model evaluation discipline through grouped data splits, bootstrap intervals, holdout restrictions, subgroup guardrails and explicit rejection of candidates that fail pre-registered criteria.
- Appreciation of responsible AI requirements, including privacy-preserving image capture, transparent assumptions, source attribution, abstention and human oversight.

- Improved project collaboration through role allocation across research/data collection and backend/frontend development, with integration dependencies managed through shared contracts and configuration.

REFERENCES

[1] N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," arXiv:2408.00714, 2024. [Online]. Available: https://arxiv.org/abs/2408.00714.

[2] M. Tschannen et al., "SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features," arXiv:2502.14786, 2025. [Online]. Available: https://arxiv.org/abs/2502.14786.

[3] R. Wang et al., "MoGe-2: Accurate Monocular Geometry with Metric Scale and Sharp Details," arXiv:2507.02546, 2025. [Online]. Available: https://arxiv.org/abs/2507.02546.

[4] M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv:2304.07193, 2023. [Online]. Available: https://arxiv.org/abs/2304.07193.

[5] S. Garrido-Jurado et al., "Automatic generation and detection of highly reliable fiducial markers under occlusion," Pattern Recognition, vol. 48, no. 6, pp. 2051-2061, 2014.

[6] National Institute of Nutrition, Indian Food Composition Tables 2017, ICMR-NIN, Hyderabad, 2017.

[7] Ministry of Education, Government of India, PM POSHAN Scheme: Guidelines on Food and Nutrition, 2023. [Online]. Available: https://pmposhan.education.gov.in/Files/Guidelines/2023/Guidelines%20on%20PM%20P OSHAN%20SCHEME.pdf.

[8] Lion4467cat, "NutriSense: AI-assisted meal portion estimation and nutrition assessment," GitHub repository. [Online]. Available: https://github.com/Lion4467cat/NutriSense. Accessed: 30-Oct-2026.

[9] Lion4467cat, "NutriSense - Model and Service Readiness Status," project documentation, 30-Oct-2026. [Online]. Available: https://github.com/Lion4467cat/NutriSense/blob/main/docs/production-readiness.md.

[10] Lion4467cat, "NutriSense Capture Protocol (Golden Set)," project documentation. [Online]. Available: https://github.com/Lion4467cat/NutriSense/blob/main/docs/protocol.md.

[11] Lion4467cat, “NutriSense Data Collection Manual,” project documentation. [Online]. Available: https://github.com/Lion4467cat/NutriSense/blob/main/docs/data-collection-manual.md.

[12] FastAPI, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/.

[13] PyTorch, “PyTorch documentation.” [Online]. Available: https://pytorch.org/docs/stable/index.html.

[14] OpenCV, “OpenCV documentation: ArUco markers.” [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html.

[15] React, “React documentation.” [Online]. Available: https://react.dev/.

[16] Hugging Face, “Transformers documentation.” [Online]. Available: https://huggingface.co/docs/transformers/index.

APPENDIX A: SNAPSHOTS

